

REQ-006：场景求交加速结构架构说明

本文面向需要继续维护或扩展 CPU 光线追踪器的开发者，说明场景求交加速结构的架构、数据流、算法选择、线程边界和验证方式。

本文讨论的实现包含三种可选路径：

模式	规范名称	作用
暴力求交	brute_force	逐个测试场景中的全部三角形，不建立空间层次结构
包围体层次	bvh	使用 BVH 逐层裁剪不可能命中的三角形
八叉树	octree	使用空间八分递归裁剪求交范围

默认模式仍然是 brute_force，因此不传 acceleration 选项的旧调用保持原有行为。

1. 设计目标与非目标

1.1 设计目标

本次实现需要同时满足以下目标：

- 保留原有暴力求交，作为兼容路径和正确性基准。
- 为同一份 world-space 三角形数据提供 BVH 和 Octree 两种加速结构。
- 让调用方可以在同步、异步、Full、Tile、Pixel 渲染中选择结构。
- 三种结构在命中结果上保持一致，包括 t、法线、UV、材质 ID 和三角形索引。
- 结构构建完成后允许多个渲染线程并发只读求交。
- 非法选项在渲染启动前被拒绝，不能静默降级为另一种结构。

1.2 非目标

本次没有实现以下功能：

- 把 BVH 或 Octree 序列化到场景文件；
- 在场景发生局部变化时增量更新结构；
- SAH（Surface Area Heuristic）BVH 优化；
- 三角形裁剪、复制或更复杂的空间分割；
- GPU 加速结构或 GPU 求交；
- 多个渲染任务之间共享结构缓存。

结构当前按每次渲染请求重新构建。这种设计让请求快照和结构生命周期简单、确定且容易与异步线程边界配合。

2. 整体架构

场景提取、结构选择、结构构建和并发求交之间的关系如下：





实现分为四层：

层	主要文件	职责
接口层	src/accel/accel_interface.h	定义 build、intersect、intersect_p 语义
公共几何层	src/accel/aabb.*、src/accel/triangle_intersection.*	AABB、slab 求交、Moller-Trumbore 求交和 Hit 填充
结构实现层	brute_force_accel.*、bvh_accel.*、octree_accel.*	三种场景求交实现
选择和管线层	acceleration_structure.*、raytrace_exporter.cpp	解析选项、创建结构、接入同步/异步渲染

3. 核心接口与所有权

3.1 AccelInterface

接口位于 src/accel/accel_interface.h：

```
class AccelInterface {
public:
    virtual ~AccelInterface() = default;

    virtual void build(const Scene& scene) = 0;
    virtual bool intersect(const Ray& ray, Hit* hit,
        real_t t_max = Math_INF) const = 0;
    virtual bool intersect_p(const Ray& ray,
        real_t t_max = Math_INF) const;
};
```

接口表达了一个重要生命周期：

- 1. 先用 build(scene) 建立结构；
- 2. 再使用 const 的求交接口；
- 3. 一次渲染结束或下一次请求开始时才销毁或替换结构。

AccelInterface 不持有 Godot 场景节点，只持有渲染器自己的值数据。结构的创建者是当前 CpuPathTracer，其生命周期由 tracer 的 std::unique_ptr<AccelInterface> 管理。

3.2 Scene 与加速结构的数据复制

Scene 仍然只保存：

- `std::vector<Triangle> triangles;`
- `std::vector<Material> materials;`
- `std::vector<Light> lights。`

BVH、Octree 和暴力结构在 `build()` 时复制 `scene.get_triangles()`。因此构建后：

- worker 不需要再读取 Godot 场景树；
- 原始 Scene 即使离开提取函数，也不会使结构中的三角形悬空；
- 求交线程只读取结构内部的 `std::vector`。

代价是每个请求会有一份三角形复制，以及一份对应结构的索引或节点存储。

4. 公共求交语义

三种结构共享 `src/accel/triangle_intersection.*`，避免不同实现逐渐产生数值语义分叉。

4.1 三角形求交

`intersect_triangle()` 使用 Moller-Trumbore 算法，输入为 world-space Triangle 和 Ray，输出为 `t`、`u`、`v`。

保持以下约束：

- 行列式绝对值小于 `DET_EPSILON` 时视为平行或退化；
- `t <= T_MIN` 的命中被拒绝，用于避免自相交；
- `t > t_max` 的命中被拒绝；
- `u < 0`、`v < 0` 或 `u + v > 1` 的结果被拒绝。

4.2 Hit 填充

`fill_triangle_hit()` 使用 `u`、`v` 和 `w = 1 - u - v` 插值：

- position: `ray(t)`；
- normal: 插值顶点法线并归一化；
- UV: 插值三角形 UV；
- materialId: 来自三角形；
- triangleIndex: 结构遍历过程中保存的原始三角形索引；
- wo: `-ray.d`。

如果顶点法线插值结果为零，则回退到几何法线。这样结构实现只负责找到三角形和重心坐标，Hit 字段的生成逻辑保持统一。

4.3 最近命中与相同距离

暴力结构原本按顺序遍历三角形，距离相同的情况下后出现的三角形会覆盖先出现的结果。共享的 `is_better_triangle_hit()` 保持了这个规则：

```
candidate_t < closest_t
或 candidate_t == closest_t 且 candidate_index > closest_index
```

BVH 和 Octree 必须使用同一规则，否则空间遍历顺序改变时会导致输出图像或材质选择发生差异。

5. AABB 与 slab 求交

`src/accel/aabb.*` 提供：

- `triangle_bounds()`：计算三角形 AABB；

- `union_bounds()`: 合并两个 AABB;
- `bounds_centroid()`、`bounds_extent()`: 计算中心和尺寸;
- `bounds_contains()`: 判断一个包围盒是否完全包含另一个包围盒;
- `intersect_bounds()`: 使用 slab 算法判断射线是否进入包围盒。

对每个坐标轴，求射线进入和离开该轴 slab 的参数区间，并不断求交：

```
t_enter = max(各轴进入参数, 0)
t_exit = min(各轴离开参数, t_max)
```

当 $t_{\text{enter}} > t_{\text{exit}}$ 时，射线不可能在 $[0, t_{\text{max}}]$

范围内穿过包围盒。方向分量接近零时，不进行除法，而是检查射线原点是否位于该轴 slab 内。

6. 暴力求交

实现位于 `src/accel/brute_force_accel.cpp`。

构建阶段只复制三角形，求交阶段逐个测试所有三角形，并维护当前最近命中：

```
closest_t = t_max
for triangle in triangles:
    if intersect_triangle(triangle, ray, closest_t):
        更新最近命中
```

它的价值不只是兼容旧行为，也有两个工程用途：

1. 作为 BVH 和 Octree 的正确性参考实现；
2. 在小场景或调试场景中避免结构构建开销。

它的复杂度是每条射线 $O(N)$ ，其中 N 为三角形数量。

7. BVH 实现

实现位于 `src/accel/bvh_accel.h/.cpp`。

7.1 节点布局

每个节点包含：

```
bounds 当前节点 AABB
left 左子节点索引，叶节点为 -1
right 右子节点索引，叶节点为 -1
first primitive_indices 中的起始位置
count 叶节点中的三角形数量
```

三角形本身存储在 `triangles`，节点只保存 `primitive_indices` 范围，不复制三角形。

7.2 构建策略

BVH 构建过程：

1. 计算当前范围内所有三角形的整体 AABB；
2. 计算三角形包围盒中心的 centroid bounds；
3. 选择最大 centroid extent 对应的坐标轴；
4. 使用 `std::nth_element` 按中心坐标做中位数划分；
5. 递归构建左右子节点；
6. 三角形数量不超过 4 时停止，形成叶节点。

如果所有三角形中心在各轴上都几乎重合，则停止继续分裂，避免无意义的递归。

这里使用中位数划分而不是

SAH，原因是实现简单、构建时间可控，并且行为确定。后续如果大型静态场景的性能成为瓶颈，可以单独引入 SAH 需求，不改变求交接口。

7.3 遍历策略

遍历一个内部节点时，先测试左右包围盒的入射距离：

1. 先遍历进入距离较小的子节点；
2. 更新当前最近命中距离；
3. 再测试另一个子节点时使用更新后的 `closest_t`，不可达的远节点会被裁剪。

这种顺序让最近命中尽早产生，从而提高后续 AABB

剪枝概率。遍历过程中不创建每条射线的堆对象，递归状态位于调用栈，临时命中值位于调用者提供的局部变量。

8. Octree 实现

实现位于 `src/accel/octree_accel.h/.cpp`。

8.1 根节点

Octree 根节点使用场景所有三角形的整体 AABB，再扩展为包围场景的立方体。立方体保证每次沿 X、Y、Z 轴二分时，八个子节点的空间尺度一致。

8.2 三角形归属规则

一个三角形只有在其 AABB

被某一个子节点完全包含时才下沉到该子节点。如果三角形跨过任一分割面，则保留在当前父节点：

完全落入一个子节点 -> 下沉一次
跨越分割面 -> 保留在父节点

这个规则避免了把一个三角形复制到多个子节点，也避免了求交时重复测试同一个三角形。代价是父节点可能保留较大三角形，空间裁剪能力不如允许复制或裁剪的高级实现，但正确性和内存行为更容易控制。

8.3 停止条件

当前实现满足任一条件就形成叶节点：

- 三角形数不超过 8；
- 深度达到 16；
- 节点最大尺寸小于 `MIN_NODE_EXTENT`。

这些上限用于避免退化场景导致无限递归或产生大量空节点。

8.4 遍历策略

Octree 先测试当前节点保留的三角形，然后收集命中的子节点入射距离，使用固定 8

项数组排序，再按近到远递归遍历。`intersect_p()`

使用提前返回：当前节点或任一子节点找到命中后立即结束，不构造完整 Hit。

9. 结构选择与 API

9.1 类型和工厂

`src/accel/acceleration_structure.*` 定义：

```
enum class AccelerationType {  
    BruteForce,  
    Bvh,  
    Octree,  
};
```

```
};
```

工厂函数负责把枚举转换为具体实现：

```
BruteForce -> BruteForceAccel
Bvh        -> BvhAccel
Octree     -> OctreeAccel
```

调用方只依赖 AccelInterface，不需要在渲染代码中写三套分支。

9.2 options 选项

公开入口都读取：

```
{
  "acceleration": "bvh"
}
```

规范值和兼容别名如下：

输入值	解析结果
brute_force	暴力求交
none	暴力求交，兼容别名
bvh	BVH
octree	Octree
oct	Octree，兼容别名

解析会忽略大小写。未知值（例如 grid）直接返回错误，不启动渲染。结果字典和 timing log 使用规范名称，不回显别名。

9.3 编辑器插件

编辑器插件在 3D 工具栏中提供 Brute Force、BVH、Octree 下拉框。任务运行期间控件锁定，避免用户在已有请求快照后改变 UI 选择造成状态误解。

10. 同步和异步渲染生命周期

同步入口的关键顺序是：

```
parse options
-> prepare_render_request
-> 提取 Scene 和 Camera 快照
-> 保存 AccelerationType
-> tracer.reset
-> tracer.set_accel(factory(type))
-> accel->build(scene)
-> Pixel / Tile / Full 求交
-> 转换 Film、保存文件、返回 Dictionary
```

异步模式把请求快照传给任务 worker：

```
提交线程
-> 创建 RenderRequest 和 RenderJob
worker
-> tracer.reset
-> set_accel(factory(type))
-> build(scene)
-> Full 模式调度 helper 线程
-> 所有计算线程 join
-> 发布 compute_done
提交线程 poll/cancel
-> join worker
-> 检查取消
```

- > 创建 Image、保存 PNG 和 timing log
- > 设置 done=true

worker 使用 RenderRequest 中的纯值快照，不继续访问 Camera、Viewport、场景树或其他 Godot Object。加速结构本身也不调用 Godot API。

11. 多线程安全模型

11.1 可以并发的部分

结构完成 build() 后，以下操作可以由多个线程并发执行：

- 同一个 BVH 的 intersect() 和 intersect_p();
- 同一个 Octree 的 intersect() 和 intersect_p();
- 同一个暴力结构的上述操作。

理由是：

- 内部 triangles、nodes、索引数组只读；
- 每条求交调用使用自己的 closest_t、候选索引和 TriangleIntersection；
- Hit* 是调用方输出，调用方必须为不同线程提供不同对象；
- 没有依赖 Godot 场景对象或全局可变量求交状态。

11.2 不能并发的部分

以下操作必须与求交阶段隔离：

- build() 与 intersect() 并发；
- CpuPathTracer::reset() 与 render_tile() 并发；
- CpuPathTracer::set_accel() 与正在进行的求交并发；
- 多线程同时调用同一个 tracer 的 render_next_tile();
- 多个线程写同一个 Hit、同一个像素或同一个 Film 槽位。

线程安全不是通过给加速结构所有方法加锁实现的，而是通过“先构建、后只读”和上层任务调度保证的。这样避免在每条射线的热路径中引入互斥锁。

11.3 tile 独占写入

Full 模式将图像拆成 16×16 tile。调度器通过原子任务索引保证每个 tile 只领取一次；不同 tile 的像素区域不重叠。

每个 tile 的统计先在本地图计，tile

完成后再通过一次互斥归并到总统计。加速结构的求交本身不写共享统计，也不写共享 Film 数据。

11.4 多个渲染任务

多个独立异步任务各自拥有一份请求、一个 tracer、一个加速结构实例和一份 Film，因此任务之间没有共享结构的数据竞争。但每个任务都会按本机逻辑线程数申请计算线程，同时启动大量任务可能导致 CPU 过量订阅。

12. 复杂度与取舍

设三角形数量为 N，树深度为 D。

模式	构建特征	求交期望特征	主要取舍
暴力	$O(1)$ 额外结构构建	$O(N)$	最简单，适合小场景和基准

模式	构建特征	求交期望特征	主要取舍
BVH	中位数递归划分，约 $O(N \log N)$	通常显著少于 N 个三角形测试	构建简单稳定，但不是 SAH 最优
Octree	递归空间划分，最坏受深度限制	受空间分布影响	三角形不复制，跨分区大三角形会留在父节点

当前优先级是正确性、确定性和线程边界清晰，而不是对所有场景做极限性能调优。后续性能需求应使用真实场景基准决定是否调整叶节点大小、深度、包围盒膨胀或引入 SAH。

13. 测试覆盖

13.1 原生测试

文件：tests/accel/acceleration_structure_tests.cpp。

覆盖内容包括：

- 三种类型名称解析和工厂创建；
- none、oct 别名和未知选项拒绝；
- 空场景、退化三角形、平行射线和未命中；
- 有限 $t_{\max}$ ；
- 跨 Octree 分区的大三角形；
- 强制 BVH/Octree 分裂的多三角形场景；
- 最近命中和相同距离时的 triangle index；
- intersect() 与 intersect_p() 和暴力结果对比；
- Hit 的位置、法线、UV、材质 ID 和三角形索引对比。

构建和运行：

```
cmake -S . -B build/req006-tests -G "Visual Studio 17 2022" -A x64 -DBUILD_TESTING=ON
cmake --build build/req006-tests --config Debug
ctest --test-dir build/req006-tests -C Debug --output-on-failure
```

13.2 Godot 集成测试

文件：tests/godot/req006_acceleration_selection_smoke.gd。

集成测试验证：

- 三种 Full 输出逐字节一致；
- 选择项正确回显到结果 Dictionary；
- BVH Pixel 模式只生成一个主射线；
- Octree Tile 模式只渲染目标 tile；
- 异步 Octree 保留选择结果；
- timing log 记录规范结构名；
- 非法 acceleration 不启动渲染。

此外，REQ-001、REQ-002、REQ-005 的回归测试确保后处理、多线程调度和异步主线程收尾没有被结构选择改坏。

14. 推荐源码阅读顺序

第一步：理解接口

阅读 `src/accel/accel_interface.h` 和 `src/scene/rt_scene.h`，确认结构只接收渲染器自己的 `Scene`，不直接依赖 Godot 场景节点。

第二步：理解公共求交语义

阅读 `src/accel/aabb.*` 和 `src/accel/triangle_intersection.*`，重点是 `t_max`、`T_MIN`、`Hit` 插值和同距命中规则。

第三步：对比三种结构

按以下顺序阅读：

```
src/accel/brute_force_accel.cpp
src/accel/bvh_accel.cpp
src/accel/octree_accel.cpp
```

暴力结构用于理解基准循环；BVH 用于理解索引范围和近到远遍历；Octree 用于理解父节点保留策略。

第四步：理解选择入口

阅读 `src/accel/acceleration_structure.cpp` 和 `src/raytrace_exporter.cpp`，跟踪 `parse_acceleration_selection()` 到 `create_acceleration_structure()`，再看 `RenderRequest.acceleration_type` 如何进入同步和异步执行路径。

第五步：理解并发边界

阅读 `src/render/cpu_path_tracer.cpp`、`src/render/parallel_task_scheduler.cpp` 和 `src/raytrace_exporter.cpp` 中的 `render_one_pass`，确认结构只在构建阶段写入，求交阶段只读；确认 `tile` 由调度器唯一领取，所有 `helper` 线程在读取 `Film` 和统计前已经 `join`。

第六步：阅读测试

最后阅读 `tests/accel/acceleration_structure_tests.cpp` 和 `tests/godot/req006_acceleration_selection_smoke.gd`。测试是实现约束的可执行说明，尤其能帮助理解跨分区三角形、有限 `t_max` 和三种结构结果等价的要求。

15. 后续扩展指南

15.1 新增一种结构

建议按以下顺序：

1. 新建继承 `AccelInterface` 的类；
2. 复用 `triangle_intersection.*`，不要重新实现 `Hit` 填充；
3. 复用 `aabb.*`，除非新结构确实需要不同包围体；
4. 在 `AccelerationType` 增加枚举；
5. 在名称解析、名称输出和工厂中同步增加分支；
6. 将选项贯穿 `RenderRequest`、`RenderOutcome`、结果 `Dictionary` 和 `timing log`；
7. 增加与暴力结构逐项比较的原生测试；
8. 增加同步、异步和非法选项的 Godot 集成覆盖。

15.2 修改节点或遍历策略

修改时必须重新确认：

- `build` 完成后节点数组是否完全只读；
- 求交是否仍然使用 `t_max` 剪枝；

- 同距命中是否仍然选择相同的 triangle index;
- intersect_p() 是否可以提前返回且不填充无效 Hit;
- 是否在每条射线热路径引入了堆分配或锁;
- 空场景、退化几何和极小包围盒是否仍然安全。

15.3 动态场景

如果未来需要支持 Mesh 动态变换或局部修改，不应直接在求交线程中修改现有结构。建议重新构建新结构后，以一次受控的所有权切换替换旧结构，或另立需求设计增量更新和读写并发协议。在没有明确生命周期协议前，禁止让 build() 与并发求交共享同一实例。

16. 已知限制

1. 结构不跨渲染请求缓存，每次请求都重新复制三角形并构建。
2. BVH 使用确定性中位数划分，不保证 SAH 意义上的最优树。
3. Octree 不复制跨分区三角形，因此大三角形可能在父节点被多个射线检查。
4. 多个异步任务同时运行时，线程数按任务分别计算，可能过量订阅 CPU。
5. 当前验证重点是命中等价性和线程安全；不同场景的性能参数仍需独立基准需求。

17. 相关文档和记录

- REQ-006 完成记录
- REQ-002 多线程光线追踪
- REQ-005 异步导出线程边界
- 项目加速结构源码